

FinDoc QA

Financial Document Question Answering System

End-to-End RAG Pipeline with Hybrid Retrieval and Systematic Evaluation

Wanrong (Chloe) Dang
M.A. Statistics, Columbia University
April 2026

Component	Technology
Document Parsing	PyMuPDF (text + table detection)
Chunking	LangChain RecursiveCharacterTextSplitter
Embedding	BAAI/bge-small-en-v1.5 (384-dim)
Vector Store	ChromaDB (cosine similarity)
Sparse Retrieval	BM25 (custom implementation)
LLM Generation	OpenAI GPT-4o-mini
Frontend	Streamlit
Evaluation	Custom pipeline (accuracy, faithfulness, relevance)

Table 1: Technology Stack Overview

Table of Contents

- 1. Introduction.....3
- 2. System Architecture.....3
- 3. Data Pipeline.....4
- 4. Retrieval Strategies.....5
- 5. Evaluation Methodology.....6
- 6. Experimental Results.....7
- 7. Ablation Studies.....8
- 8. Demo Application.....10
- 9. Discussion and Future Work.....10
- 10. Conclusion.....11

1. Introduction

Financial document analysis remains one of the most labor-intensive tasks in the investment and compliance industry. Analysts routinely spend hours sifting through SEC 10-K filings, which often span 100+ pages of dense financial tables, risk disclosures, and management commentary. The ability to automatically extract precise, citation-grounded answers from these documents has significant practical value.

This project presents **FinDoc QA**, an end-to-end Retrieval-Augmented Generation (RAG) system designed specifically for financial document question answering. The system ingests a PDF filing, parses and chunks the content, builds a searchable vector index, and generates answers with explicit source citations. Unlike generic chatbot approaches, FinDoc QA emphasizes **verifiability** (every claim is traced back to a specific page and passage) and **systematic evaluation** (accuracy, faithfulness, and relevance are quantified across controlled experiments).

The system is evaluated on Apple's FY2024 10-K filing (121 pages) using a 12-question ground truth test set covering both numerical queries (e.g., revenue, R&D; spend, net income) and factual queries (e.g., CEO identity, product categories, credit rating). Systematic ablation studies are conducted over chunk size, retrieval depth (top-K), search strategy (dense vs. hybrid), and fusion weight, yielding actionable insights for RAG system design in the financial domain.

2. System Architecture

FinDoc QA follows a modular pipeline architecture with five stages: document parsing, chunking, embedding and indexing, retrieval, and generation. Each stage is independently configurable, enabling systematic experimentation with different parameters and strategies.

2.1 Pipeline Overview

PDF Document → PyMuPDF Parser → Recursive Chunking → BGE-small Embedding → ChromaDB Index
→ Hybrid Search (BM25 + Dense) → GPT-4o-mini Generation → Citation-Grounded Answer

The pipeline is designed for single-document QA over long financial filings. The user submits a natural language question, and the system retrieves the most relevant passages using a hybrid search strategy that combines dense semantic embeddings with sparse keyword matching (BM25). The retrieved context is then passed to a large language model (GPT-4o-mini) with instructions to produce a concise, citation-grounded answer.

2.2 Design Decisions

- **BGE-small-en-v1.5** was chosen as the embedding model for its strong performance-to-size ratio (384 dimensions, 33M parameters). It runs locally on CPU in under 7 seconds for 721 chunks, eliminating API costs for the embedding stage.
- **ChromaDB** was selected as the vector store for its simplicity, persistent storage support, and native cosine similarity search. For a single-document system, it provides sufficient performance

without the operational complexity of Pinecone or Weaviate.

- **GPT-4o-mini** was chosen for generation due to its low cost (~\$0.15/1M input tokens), fast inference, and strong instruction-following on citation tasks. Total API cost for all experiments was under \$2.
- **Hybrid search** (BM25 + dense) was implemented to address the known weakness of pure dense retrieval on keyword-specific financial queries (e.g., exact numerical lookups where BM25 excels).

3. Data Pipeline

3.1 Document Parsing

The PDF parsing stage uses PyMuPDF (fitz) to extract text content page by page. Each page is annotated with metadata including page number, character count, and a boolean flag indicating the presence of tables (detected via PyMuPDF's built-in table finder). The Apple FY2024 10-K yielded 121 pages with 411,031 total characters, of which 40 pages contained tabular data.

Metric	Value
Total Pages	121
Total Characters	411,031
Pages with Tables	40 (33.1%)
File Size	1.0 MB
Source	Apple Inc. FY2024 10-K Annual Report

Table 2: Document Statistics

3.2 Chunking Strategy

Text is split using LangChain's RecursiveCharacterTextSplitter with a hierarchical separator list: double newlines, single newlines, sentence boundaries, spaces, and character-level splits. This preserves paragraph-level coherence whenever possible. The baseline configuration uses chunk_size=800 and chunk_overlap=200, producing 721 chunks with an average length of 694 characters.

The impact of chunk size on downstream QA accuracy is a key research question addressed in the ablation studies (Section 7). Financial documents present a unique chunking challenge: tables, footnotes, and multi-line financial statements must be kept intact to preserve meaning. Larger chunks (1200 chars) were found to dramatically improve accuracy on numerical queries by keeping full table rows within a single chunk.

Chunk Size	Overlap	Num Chunks	Avg Length
500	100	1,008	~420 chars
800 (baseline)	200	721	694 chars
1200	300	477	~1,020 chars

Table 3: Chunking Configurations Tested

3.3 Embedding and Indexing

Each text chunk is encoded into a 384-dimensional vector using the BAAI/bge-small-en-v1.5 sentence transformer model. The model runs locally on an Apple M4 CPU, embedding all 721 baseline chunks in 6.8 seconds. Vectors are stored in ChromaDB with cosine similarity as the distance metric. Metadata

(page number, table presence, chunk index) is stored alongside each vector for post-retrieval filtering and citation generation.

4. Retrieval Strategies

4.1 Dense Retrieval (Baseline)

The baseline retrieval strategy encodes the user query using the same BGE-small model and performs a cosine similarity search against the ChromaDB index, returning the top-K most similar chunks. This approach excels at semantic matching (e.g., 'How much money did Apple make?' correctly retrieves revenue tables) but can miss specific keyword matches (e.g., 'net income' may retrieve 'net sales' or 'comprehensive income' instead).

4.2 BM25 Sparse Retrieval

A custom BM25 implementation provides term-frequency-based sparse retrieval. BM25 scores are computed using the standard formula with $k1=1.5$ and $b=0.75$. The BM25 index is built over the same chunk corpus, enabling exact keyword matching that complements the semantic understanding of dense retrieval.

4.3 Hybrid Search (BM25 + Dense)

The hybrid approach combines dense and sparse retrieval scores using a weighted linear combination:

$$score(q, d) = w_{dense} \times sim_{dense}(q, d) + (1 - w_{dense}) \times score_{BM25}(q, d)$$

Both scores are normalized to $[0, 1]$ before fusion. The dense weight w_{dense} is a tunable hyperparameter; our ablation study (Section 7.2) found that $w_{dense} = 0.6$ yields the best accuracy, indicating that semantic understanding should dominate but keyword matching provides a meaningful boost for specific financial queries.

A key finding is that hybrid search resolved a specific failure case of pure dense retrieval: the query 'What was Apple's net income in 2024?' failed under dense-only search because the most similar chunks contained 'net sales' and 'comprehensive income' rather than 'net income.' BM25's exact term matching pushed the correct chunk (Consolidated Statements of Comprehensive Income) into the top results.

5. Evaluation Methodology

5.1 Ground Truth Test Set

A 12-question ground truth test set was manually curated from the Apple FY2024 10-K filing, covering two categories: numerical queries (8 questions requiring specific financial figures) and factual queries (4 questions about company attributes). Each question has a gold answer and a set of expected keywords for automated evaluation.

Category	Count	Example
Numerical	8	What was Apple's total net sales in FY2024?
Factual	4	Who is Apple's CEO?
Total	12	

Table 4: Ground Truth Test Set Composition

5.2 Evaluation Metrics

- **Keyword Accuracy:** Whether the generated answer contains at least one expected keyword from the gold answer. This is a strict binary metric that rewards precise extraction of financial figures.
- **Faithfulness:** The proportion of numerical claims in the answer that can be traced back to the retrieved source documents. Computed by extracting all numbers from the answer and checking their presence in the source texts. A score of 1.0 indicates all cited numbers appear in the retrieved context.
- **Relevance:** The average cosine similarity (1 - distance) between the query and the top-K retrieved chunks. This measures retrieval quality independent of generation quality.

6. Experimental Results

6.1 Baseline Performance

The baseline configuration (dense retrieval, chunk_size=800, top_k=5) achieves 83.3% keyword accuracy (10/12 questions correct). All 4 factual questions were answered correctly (100%), while numerical accuracy was 75.0% (6/8). The two failures were 'net income' (retrieval missed the correct chunk) and 'total number of employees' (information scattered across multiple pages).

Metric	Value
Overall Accuracy	83.3% (10/12)
Numerical Accuracy	75.0% (6/8)
Factual Accuracy	100.0% (4/4)
Avg Faithfulness	0.619
Avg Relevance	0.735
Total Tokens	14,628
API Cost	~\$0.01

Table 5: Baseline Performance (Dense, chunk=800, top_k=5)

6.2 Dense vs. Hybrid Search

Hybrid search (dense_weight=0.6) improved overall accuracy to 91.7% (11/12), resolving the net income failure case. However, at the default weight of 0.5, hybrid search actually degraded performance to 75.0% (9/12) by over-weighting BM25 matches that displaced more semantically relevant chunks. This highlights the importance of weight tuning in hybrid retrieval systems.

Strategy	Accuracy	Net Income	CEO	Shareholder Returns
Dense Only	83.3%	FAIL	PASS	PASS
Hybrid (w=0.5)	75.0%	PASS	FAIL	FAIL
Hybrid (w=0.6)	91.7%	PASS	PASS	PASS

Table 6: Dense vs. Hybrid Search Performance on Key Questions

7. Ablation Studies

7.1 Chunk Size x Top-K (Dense Retrieval)

A systematic ablation was conducted over 9 configurations: 3 chunk sizes (500, 800, 1200) crossed with 3 top-K values (3, 5, 8). Each configuration was evaluated on the full 12-question test set using dense retrieval.

Chunk Size	Top-K	Chunks	Accuracy	Faithfulness	Relevance	Tokens
500	3	1,008	58.3%	0.413	0.753	5,954
500	5	1,008	66.7%	0.592	0.743	9,103
500	8	1,008	75.0%	0.631	0.734	13,492
800	3	721	83.3%	0.619	0.744	9,329
800	5	721	83.3%	0.622	0.735	14,610
800	8	721	91.7%	0.563	0.725	22,326
1200	3	477	83.3%	0.494	0.737	12,842
1200	5	477	100.0%	0.452	0.728	20,948
1200	8	477	100.0%	0.566	0.718	31,574

Table 7: Chunk Size x Top-K Ablation Results (Dense Retrieval)

Key findings from the chunk size ablation:

- **Chunk size dominates accuracy:** Increasing chunk size from 500 to 1200 improved accuracy from 58-75% to 83-100%, regardless of top-K. Larger chunks preserve complete table rows and multi-line financial statements, which is critical for numerical QA.
- **Top-K provides diminishing returns:** Going from top_k=3 to top_k=5 consistently improved accuracy, but top_k=8 offered marginal gains at significantly higher token cost (+50-100%).
- **Optimal configuration:** chunk_size=1200, top_k=5 achieves 100% accuracy at a moderate token cost of 20,948 tokens per evaluation run. This represents the best accuracy-cost tradeoff.
- **Faithfulness inversely correlates with chunk size:** Larger chunks include more surrounding text, increasing the denominator of the faithfulness metric. This is an artifact of the metric definition, not a genuine quality decline.

7.2 Dense Weight in Hybrid Search

The fusion weight between dense and BM25 scores was swept from 0.3 to 0.9 in increments of 0.1. A clear phase transition was observed at $w=0.6$: below this threshold, accuracy drops to 75.0% as BM25 over-dominates and displaces semantically relevant chunks; at 0.6 and above, accuracy stabilizes at 91.7%.

Dense Weight	0.3	0.4	0.5	0.6	0.7	0.8	0.9
Accuracy	75.0%	75.0%	75.0%	91.7%	91.7%	91.7%	91.7%

Table 8: Hybrid Search Weight Ablation

This result suggests that dense semantic similarity should be the primary signal for financial document retrieval, with BM25 serving as a corrective boost for keyword-specific queries. The 0.6 threshold likely reflects the point where BM25 contributes enough to resolve exact-term failures (like 'net income') without overwhelming the semantic signal that correctly handles paraphrase queries.

8. Demo Application

A Streamlit-based web interface provides interactive access to the FinDoc QA system. Users can type natural language questions and receive citation-grounded answers in real time. The sidebar exposes system configuration options including search mode (dense vs. hybrid), top-K depth, and dense weight, enabling users to observe the impact of these parameters on answer quality.

8.1 Interface Features

- **Real-time QA:** Type a question and receive an answer with [Source N, Page M] citations within 2-3 seconds.
- **Configurable retrieval:** Toggle between Dense Only and Hybrid search modes; adjust top-K and dense weight via sliders.
- **Transparency metrics:** Each answer displays Sources Used, Average Relevance score, and Tokens Used.
- **Source inspection:** Expandable panel shows the full text of each retrieved source chunk with page numbers and relevance scores.
- **Sample questions:** Sidebar provides clickable sample questions for quick exploration.

9. Discussion and Future Work

9.1 Limitations

- **Single-document scope:** The current system is designed for QA over a single PDF. Multi-document comparison (e.g., comparing revenue across multiple companies) would require architectural changes.
- **Table parsing:** While PyMuPDF detects table presence, the text extraction does not preserve table structure. Complex multi-column tables are linearized into text, which can confuse the LLM. A dedicated table parser (e.g., Camelot or Tabula) could improve numerical extraction accuracy.
- **Evaluation scale:** The 12-question test set, while carefully curated, is small. A larger benchmark with 50-100 questions across multiple 10-K filings would provide more robust performance estimates.
- **Cost-accuracy tradeoff:** The optimal configuration (chunk=1200, top_k=5) uses ~21K tokens per 12-question evaluation. At scale (thousands of queries), cost management becomes a consideration.

9.2 Future Directions

- **Multi-document QA:** Extend the system to ingest multiple filings and support comparative queries ('How did Apple's R&D; spend compare to Google's in 2024?').
- **Structured table extraction:** Integrate table-aware parsing to preserve row/column structure, improving accuracy on complex financial tables.

- **Reranking:** Add a cross-encoder reranking step after initial retrieval to improve precision at lower top-K values.
- **Streaming generation:** Implement streaming LLM output for faster perceived response time in the demo.
- **User-uploaded documents:** Allow users to upload their own PDFs through the Streamlit interface, with on-the-fly parsing and indexing.

10. Conclusion

FinDoc QA demonstrates that a well-designed RAG system can achieve high accuracy on financial document QA with minimal infrastructure and cost. The key contributions of this project are:

- A complete, reproducible RAG pipeline from PDF ingestion to citation-grounded generation, with all code publicly available on GitHub.
- Systematic ablation studies revealing that chunk size is the dominant factor in financial QA accuracy, with larger chunks (1200 chars) achieving 100% accuracy on the test set by preserving complete table structures.
- A hybrid search strategy (BM25 + dense) that resolves specific failure cases of pure semantic retrieval, with an empirically determined optimal fusion weight of 0.6.
- A custom evaluation framework measuring keyword accuracy, faithfulness, and relevance, providing quantitative evidence for design decisions rather than relying on qualitative assessment.

The total development time was approximately 3 weeks, with API costs under \$2 for all experiments. The system runs entirely on a consumer laptop (Apple MacBook Air M4) for parsing, embedding, and indexing, requiring only API access for the generation stage. This demonstrates that production-quality RAG systems for domain-specific QA are accessible to individual practitioners without enterprise infrastructure.

GitHub: github.com/dangchloe002-art/findoc-qa